深度学习基础理论

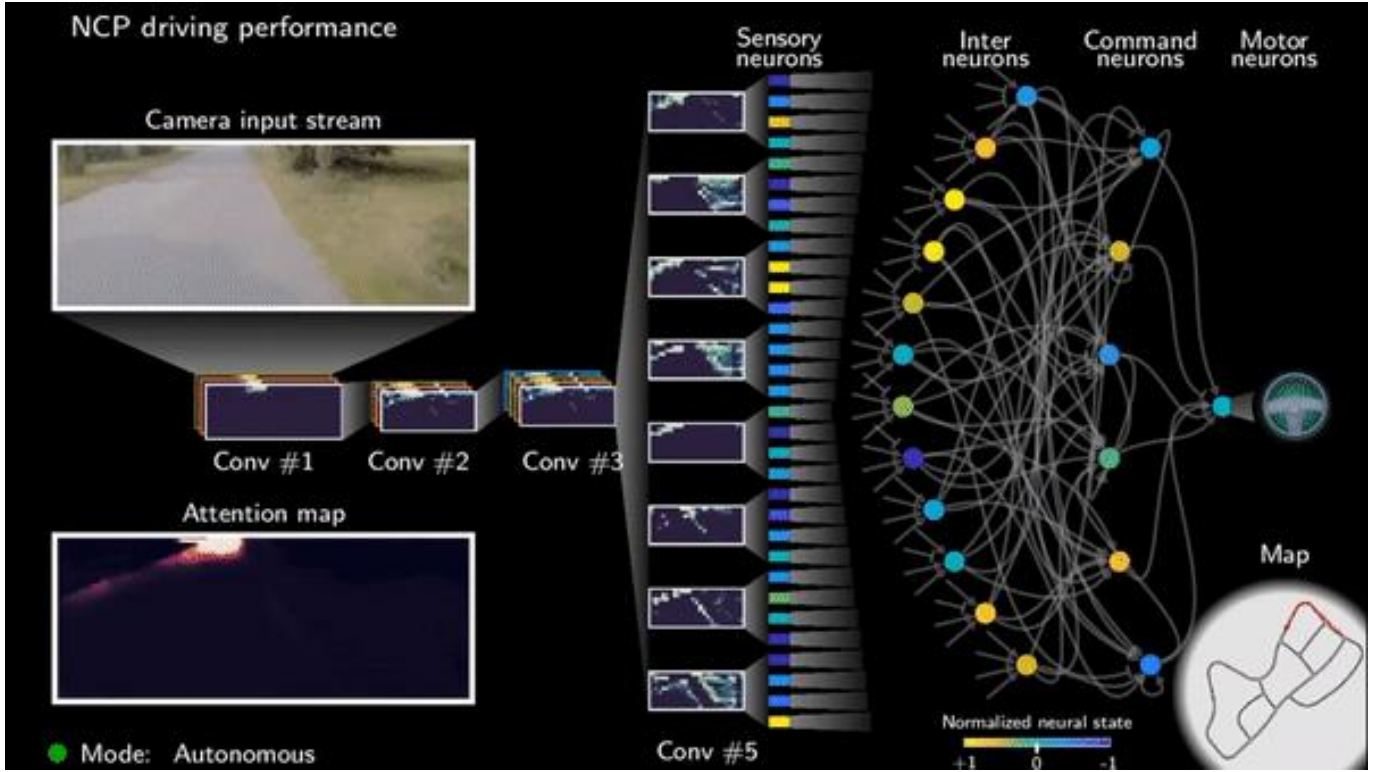
兰州大学 雍宾宾
yongbb@lzu.edu.cn

神经元模型

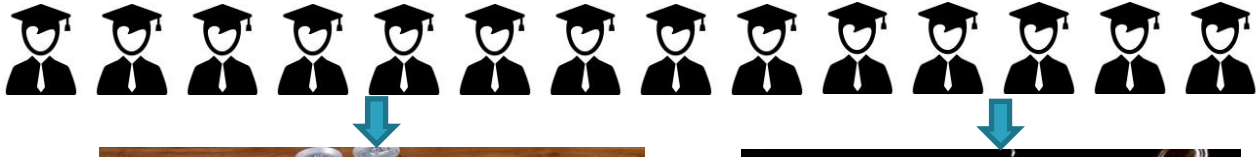
https://www.nature.com/articles/s42256-020-00237-3

2020

- 19个控制神经元
- 受到线虫线虫接线图的启发
- 单个单元内的信号处理与传统深度学习有所不同。



什么是神经网络?



× 3.0

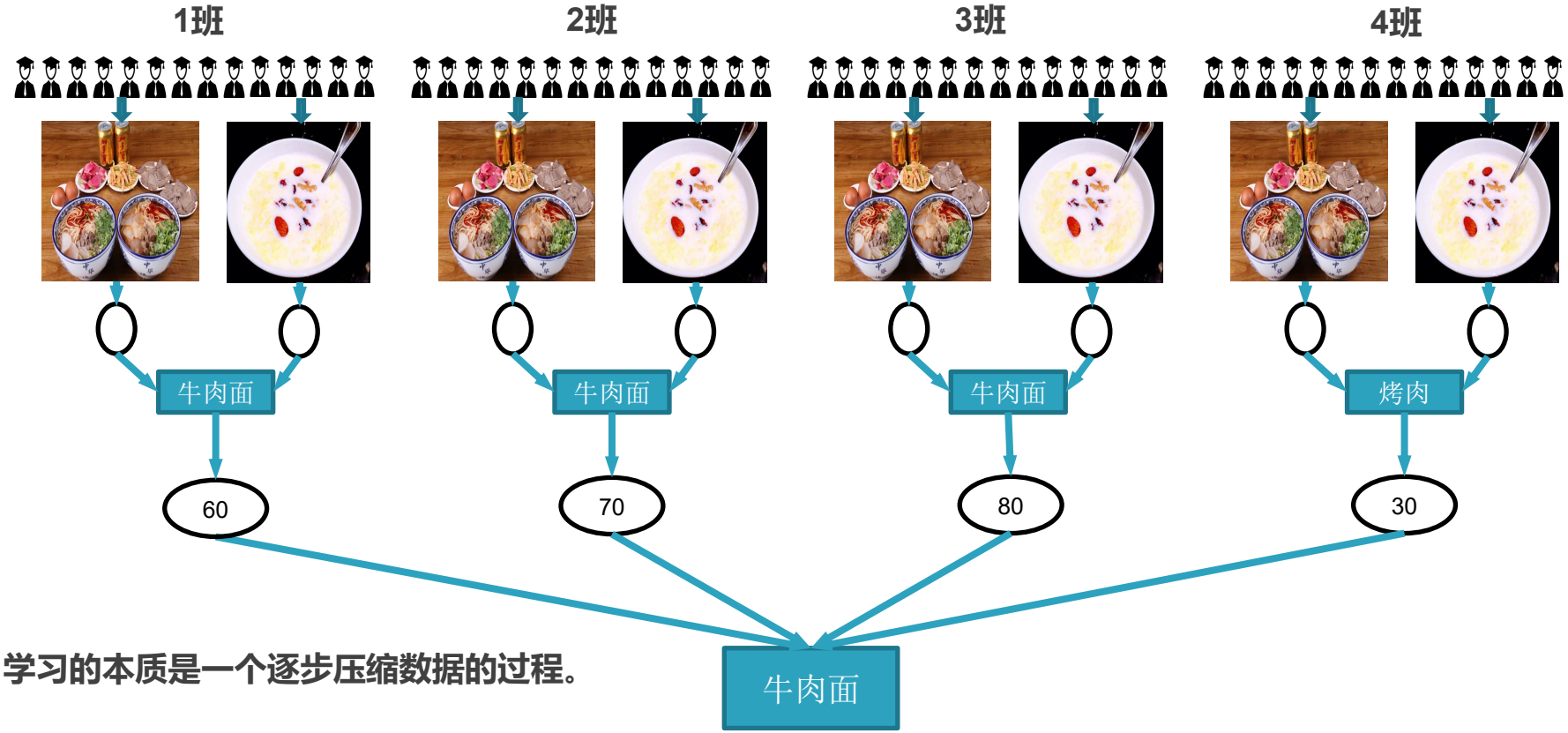
60

40

3

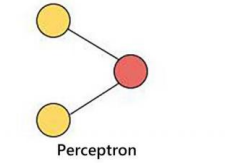
牛肉面

什么是神经网络?

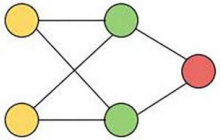


学习的本质是一个逐步压缩数据的过程。

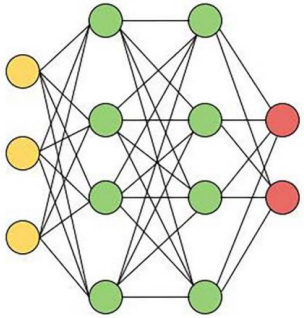
神经元到深度神经网络



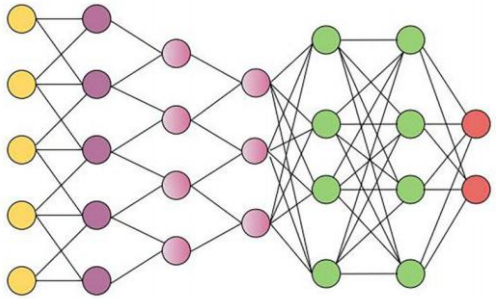
Perceptron



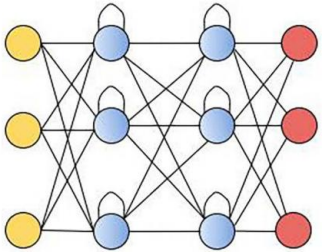
Feed Forward Network



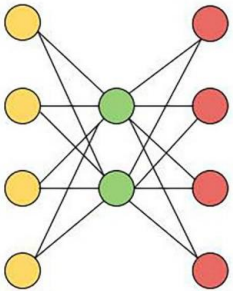
Deep Feed Forward Network



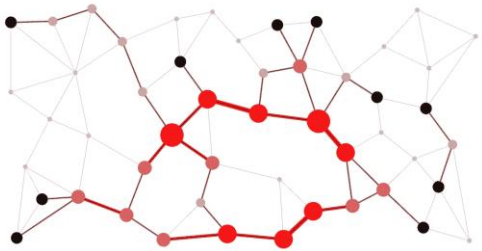
Deep Convolutional Neural Network



LSTM Network



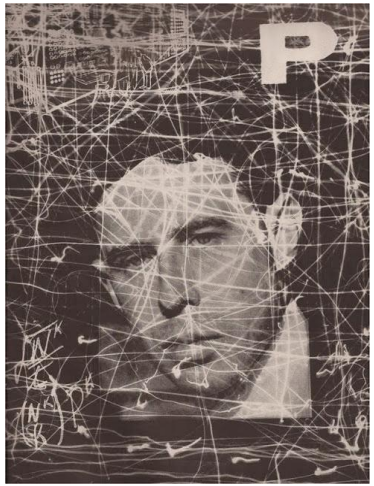
Autoencoder

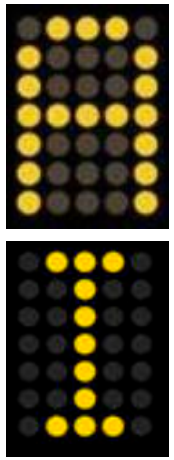


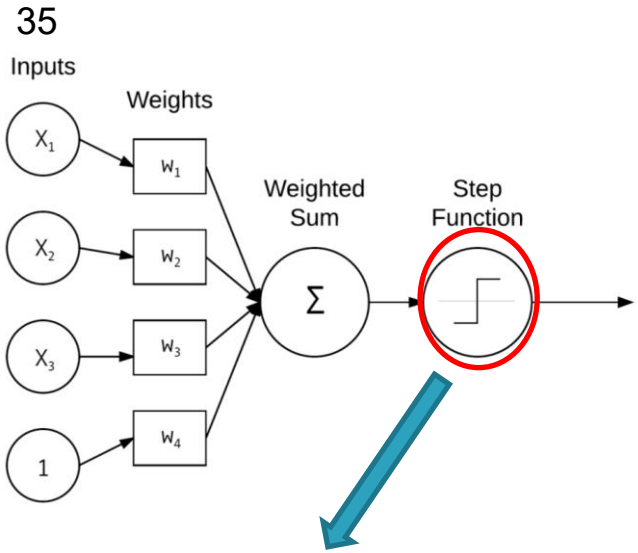
GNN

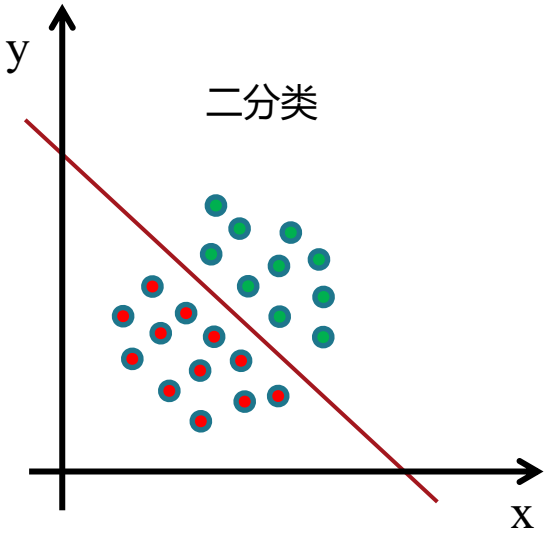
感知机模型-1958年

Frank Rosenblatt









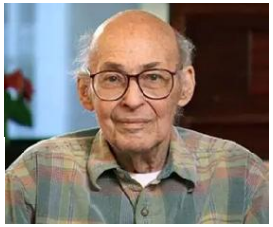
知识点1：激活函数非线性

$$\text{output} = \begin{cases} 0 & \text{if } \sum_j w_j x_j \leq \text{threshold} \\ 1 & \text{if } \sum_j w_j x_j > \text{threshold} \end{cases}$$

感知机-逻辑表示

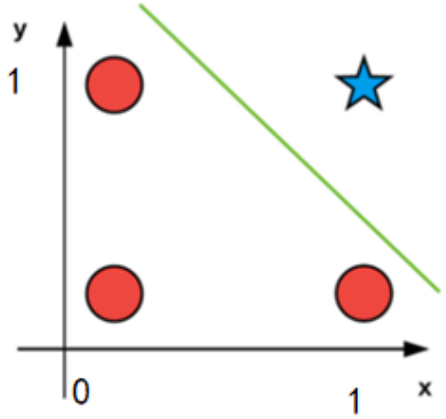
x: 我喜欢跑步
y: 我喜欢游泳
x&y: 我喜欢跑步且我喜欢游泳

x: 我喜欢跑步
y: 我喜欢游泳
x|y: 我喜欢跑步或我喜欢游泳



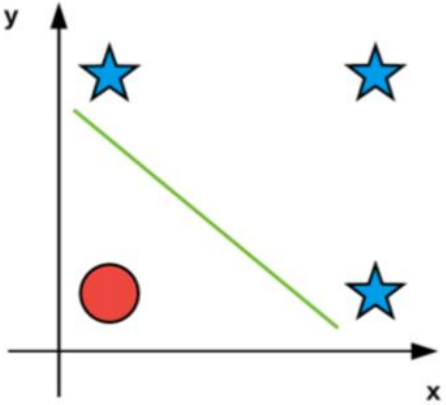
马文 明斯基
《感知机》-1969年

AND



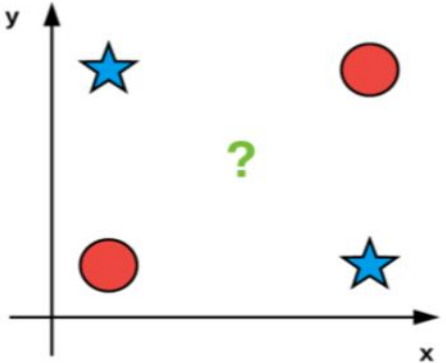
0 & 0 = 0
0 & 1 = 0
1 & 0 = 0
1 & 1 = 1

OR



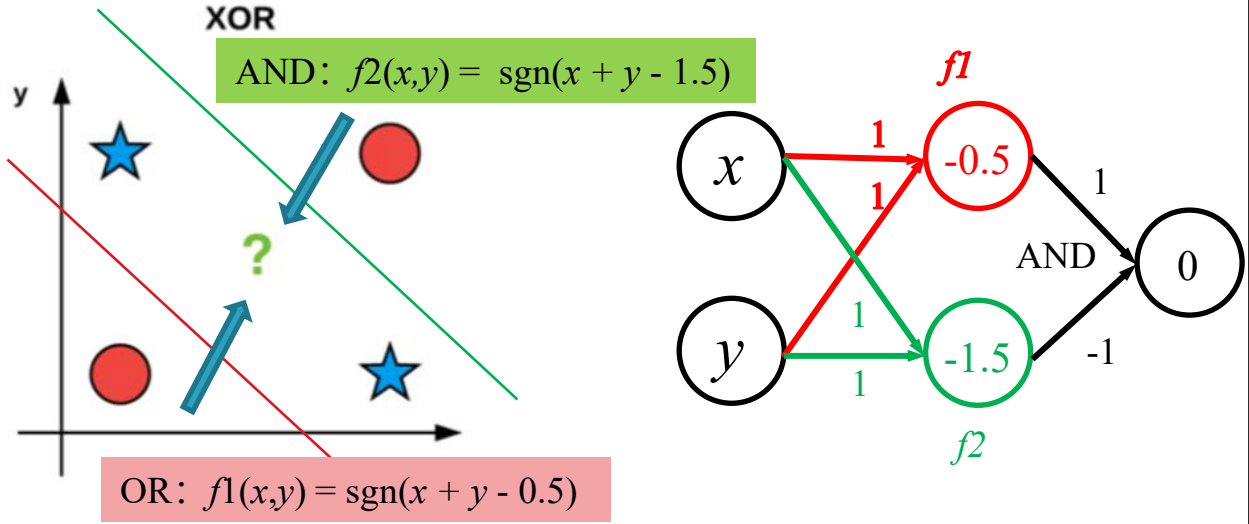
0 | 0 = 0
0 | 1 = 1
1 | 0 = 1
1 | 1 = 1

XOR



0 ^ 0 = 0
0 ^ 1 = 1
1 ^ 0 = 1
1 ^ 1 = 0

异或问题-感知机

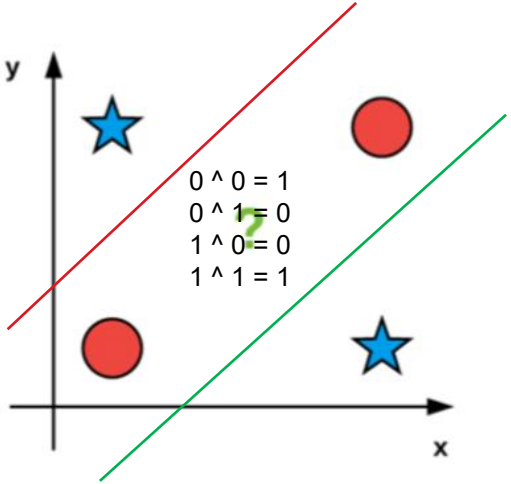


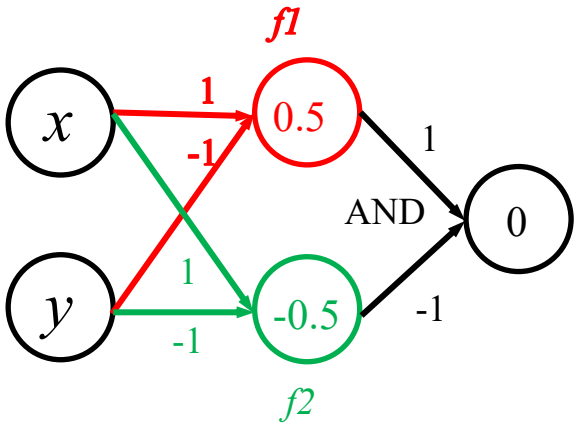

```python
import numpy as np
def step(x):
    return (np.sign(x)+1)//2

def f1(x,y):
    return step(x+y-0.5)

def f2(x,y):
    return step(x+y-1.5)

def xor(x,y):
    return step(f1(x,y)-f2(x,y))

x = np.array([0,0,1,1])
y = np.array([0,1,0,1])
z = xor(x,y)
for i in range(len(x)):
    print("xor(%d,%d)=%d" % (x[i],y[i],z[i]))
```

知识点：深层网络比浅层网络拥有更强的表达能力。

推广到同或-感知机

AND：$f1(x,y) = \text{sgn}(x - y + 0.5)$





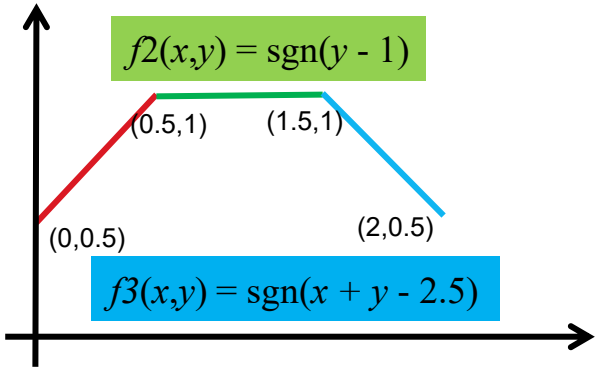
OR：$f2(x,y) = \text{sgn}(x - y - 0.5)$

```python
import numpy as np
def step(x):
    return (np.sign(x)+1)//2

def f1(x,y):
    return step(x-y+0.5)

def f2(x,y):
    return step(x-y-0.5)

def xor(x,y):
    return step(f1(x,y)-f2(x,y))

x = np.array([0,0,1,1])
y = np.array([0,1,0,1])
z = xor(x,y)
for i in range(len(x)):
    print("xor(%d,%d)=%d" % (x[i],y[i],z[i]))
```

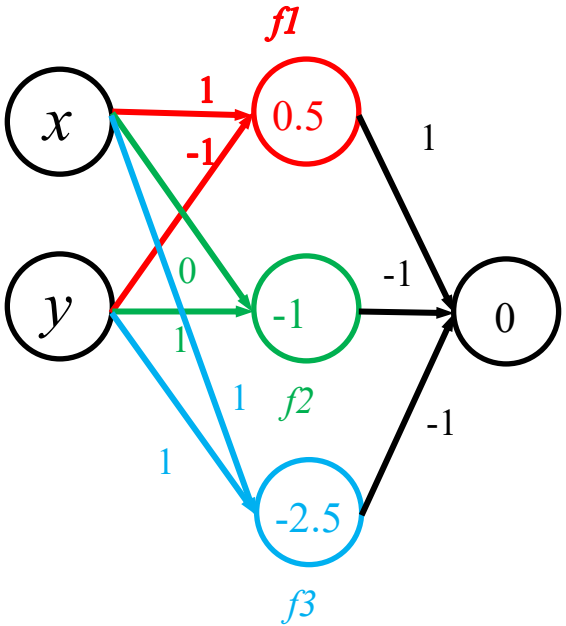
推广-感知机

$f1(x,y) = \text{sgn}(x - y + 0.5)$

$f2(x,y) = \text{sgn}(y - 1)$

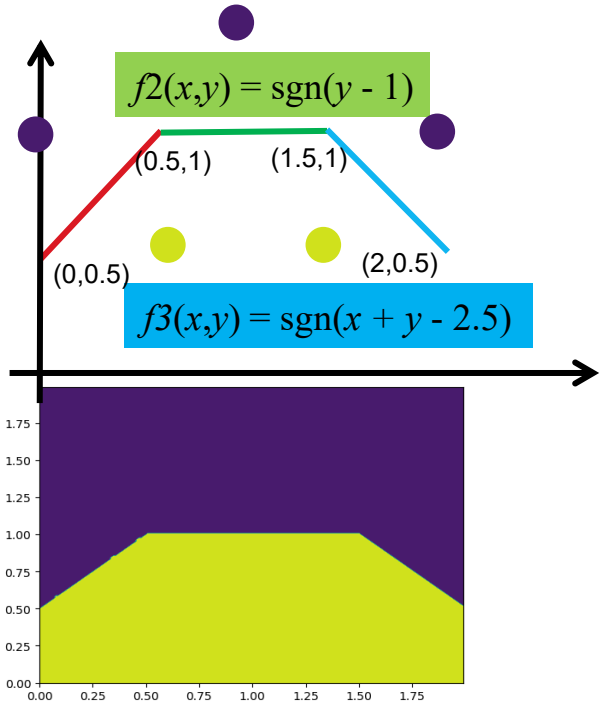


$f3(x,y) = \text{sgn}(x + y - 2.5)$



推广-感知机

$f1(x,y) = \text{sgn}(x - y + 0.5)$




```
import numpy as np
def step(x):
    return (np.sign(x)+1)//2

def f1(x,y):
    return step(x-y+0.5)

def f2(x,y):
    return step(y-1.0)

def f3(x,y):
    return step(x+y-2.5)

def clf(x,y):
    return step(f1(x,y)-f2(x,y)-f3(x,y))

x = np.array([0.5,1.5,0,1.0,2])
y = np.array([0.5,0.5,1,1.5,1])
z = clf(x,y)
for i in range(len(x)):
    print("clf(%f,%f)=%d" % (x[i],y[i],z[i]))
```

```
import matplotlib.pyplot as plt
xx, yy = np.meshgrid(np.arange(0, 2, 0.01),
np.arange(0, 2, 0.01))
input = np.c_[xx.ravel(), yy.ravel()]
zz=clf(input[:,0],input[:,1]).reshape(xx.shape)
plt.contourf(xx,yy,zz)
plt.show()
```

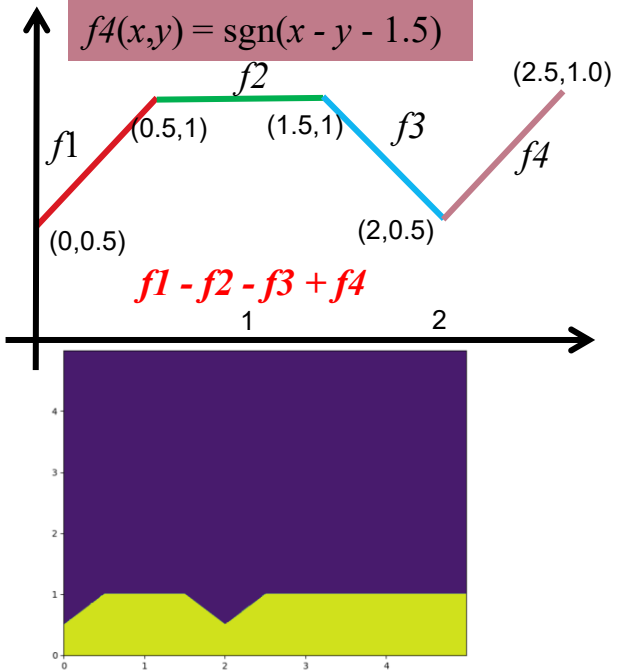
推广-感知机

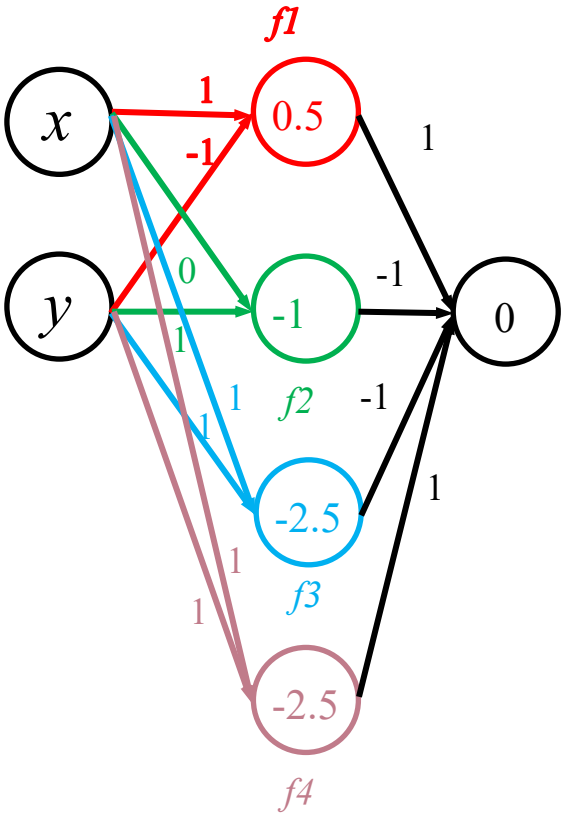
$f1(x,y) = \text{sgn}(x - y + 0.5)$

$f2(x,y) = \text{sgn}(y - 1)$

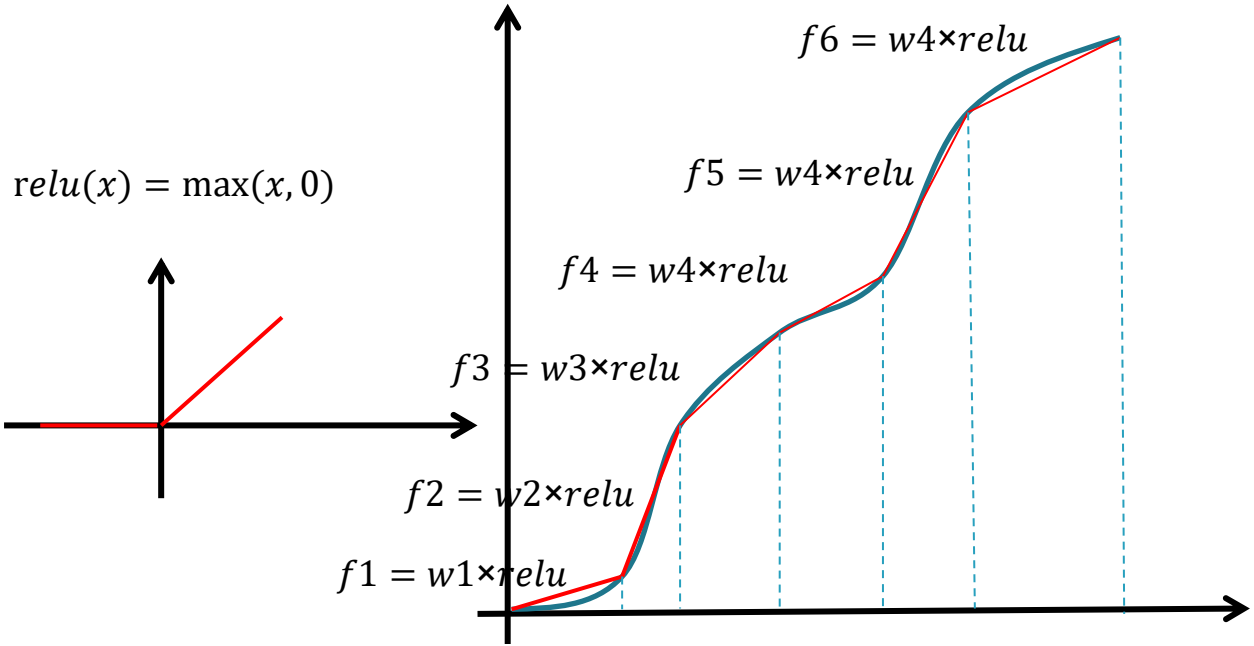
$f3(x,y) = \text{sgn}(x + y - 2.5)$

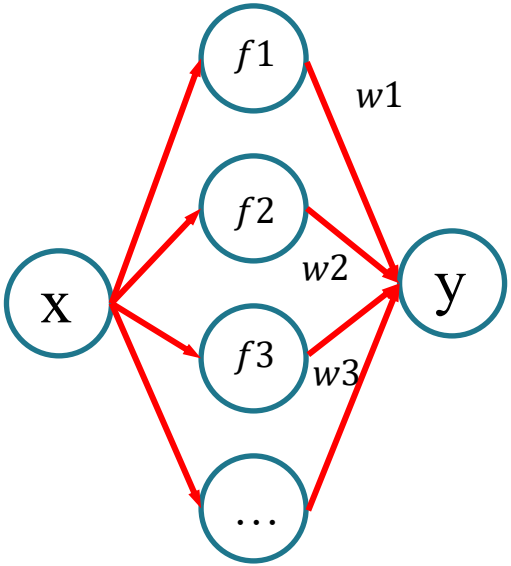
$f4(x,y) = \text{sgn}(x - y - 1.5)$



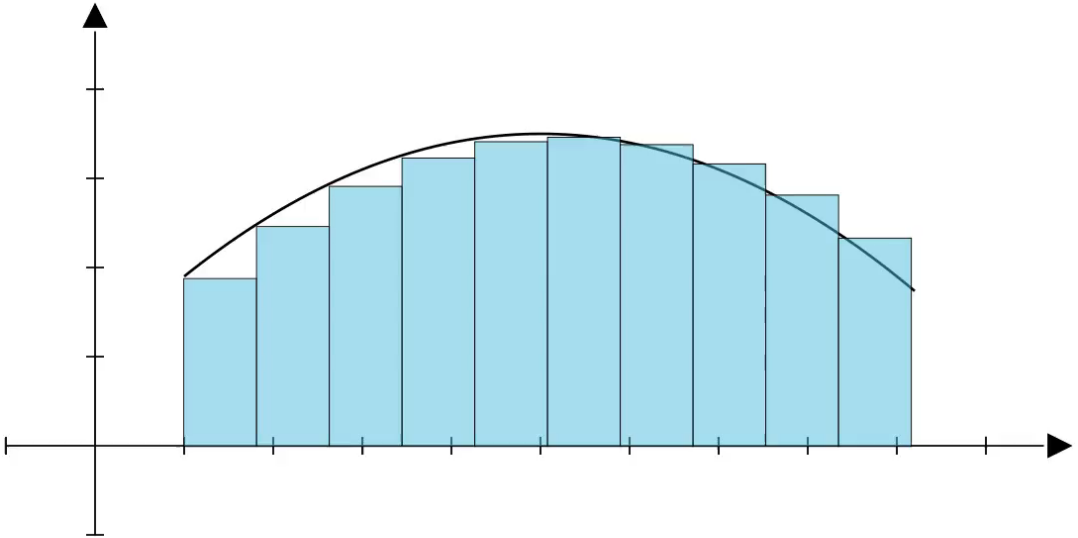


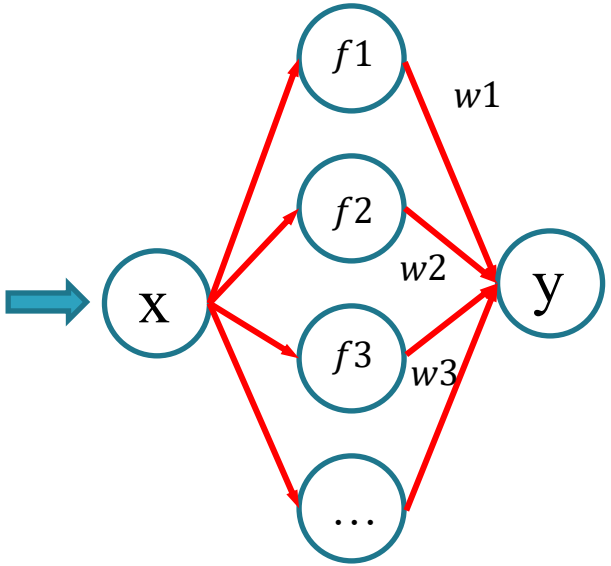
通用拟合定理


$relu(x) = \max(x, 0)$
$f6 = w4 \times relu$
$f5 = w4 \times relu$
$f4 = w4 \times relu$
$f3 = w3 \times relu$
$f2 = w2 \times relu$
$f1 = w1 \times relu$


x
$f1$
$f2$
$f3$
...
$w1$
$w2$
$w3$
y

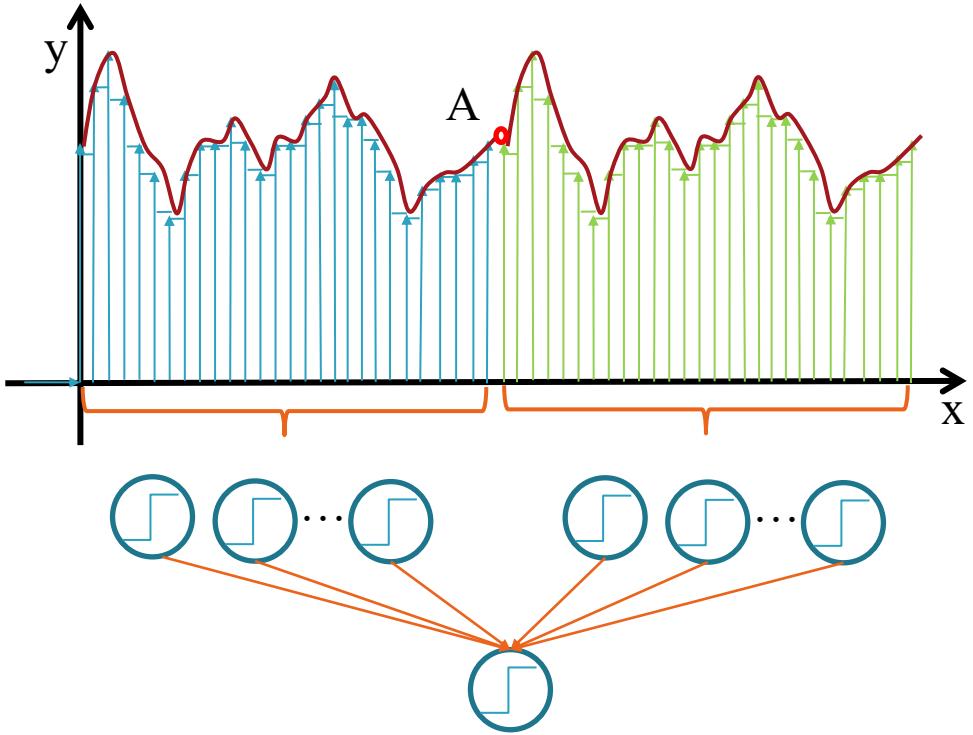
神经网络的表达能力





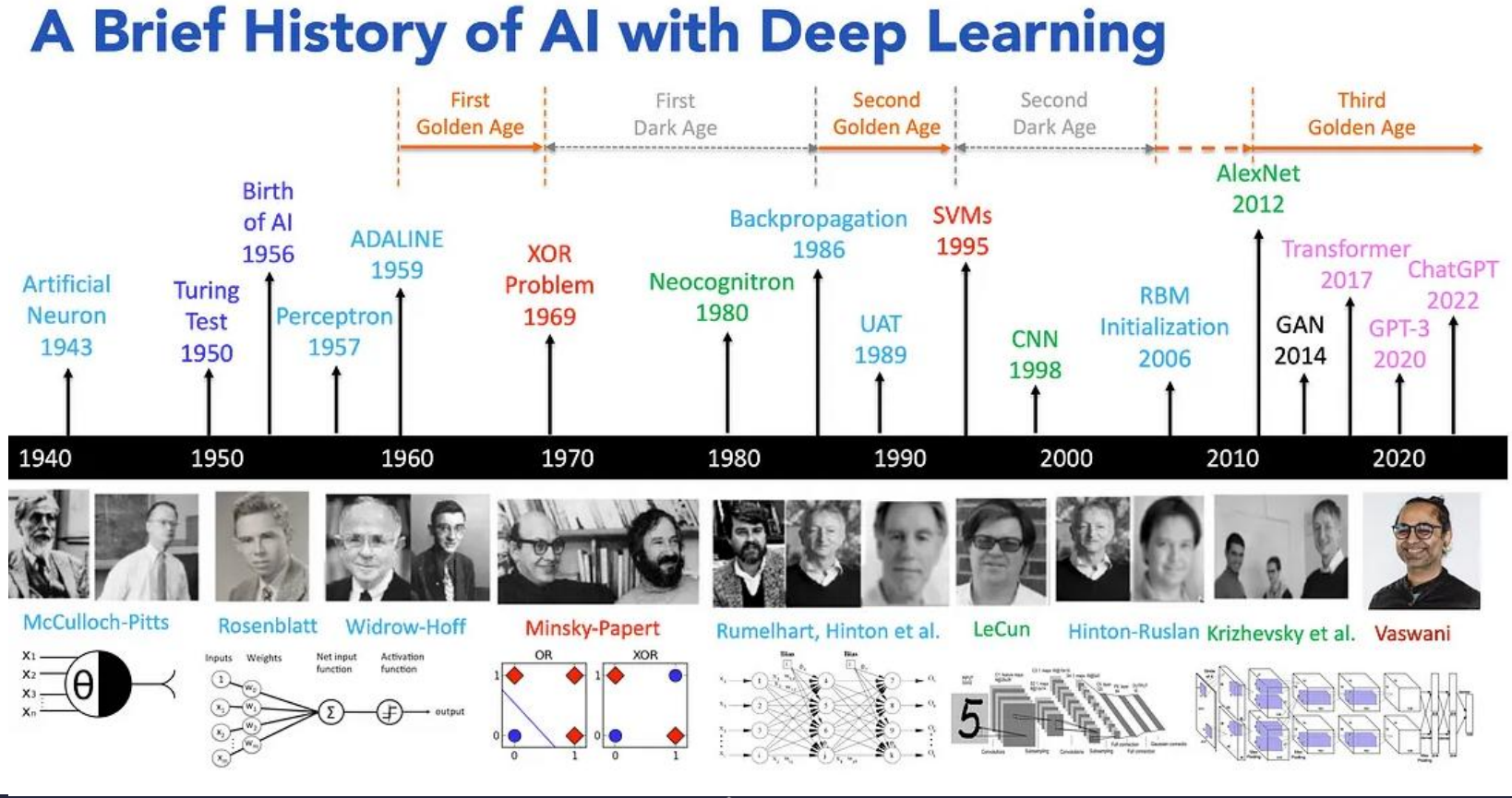
知识点：在拥有足够多隐藏层神经元的情况下：
两层神经网络(单隐含层)可以表示任意连续函数。

神经网络的表达能力



知识点：在拥有足够多隐藏层神经元的情况下，三层神经网络可以表示任意函数。

深度神经网络发展


A Brief History of AI with Deep Learning
First Golden Age
First Dark Age
Second Golden Age
Second Dark Age
Third Golden Age
Artificial Neuron 1943
Turing Test 1950
Birth of AI 1956
Perceptron 1957
ADALINE 1959
XOR Problem 1969
Neocognitron 1980
Backpropagation 1986
UAT 1989
SVMs 1995
CNN 1998
RBM Initialization 2006
AlexNet 2012
GAN 2014
Transformer 2017
GPT-3 2020
ChatGPT 2022
1940
1950
1960
1970
1980
1990
2000
2010
2020
McCulloch-Pitts
Rosenblatt
Widrow-Hoff
Minsky-Papert
Rumelhart, Hinton et al.
LeCun
Hinton-Ruslan
Krizhevsky et al.
Vaswani
Inputs
Weights
Net input function
Activation function
output
OR
XOR

知识点总结

- **激活函数非线性**
- **深层网络比浅层网络拥有更强的表达能力**
- **三层神经网络可以表示任意函数（足够隐藏单元）**

- **神经网络与图灵机(现代计算机)等价**
- **计算性能的发展带动神经网络的发展**
- **深层神经网络可以充分利用计算机的性能**

- 语音识别

$f($  $)=$ “你好”

- 图像识别

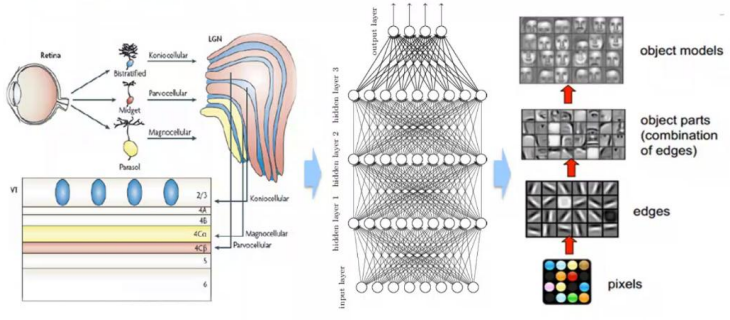
$f($  $)=$ “9”

- 围棋

$f($  $)=$ “6-5” （落子位置）

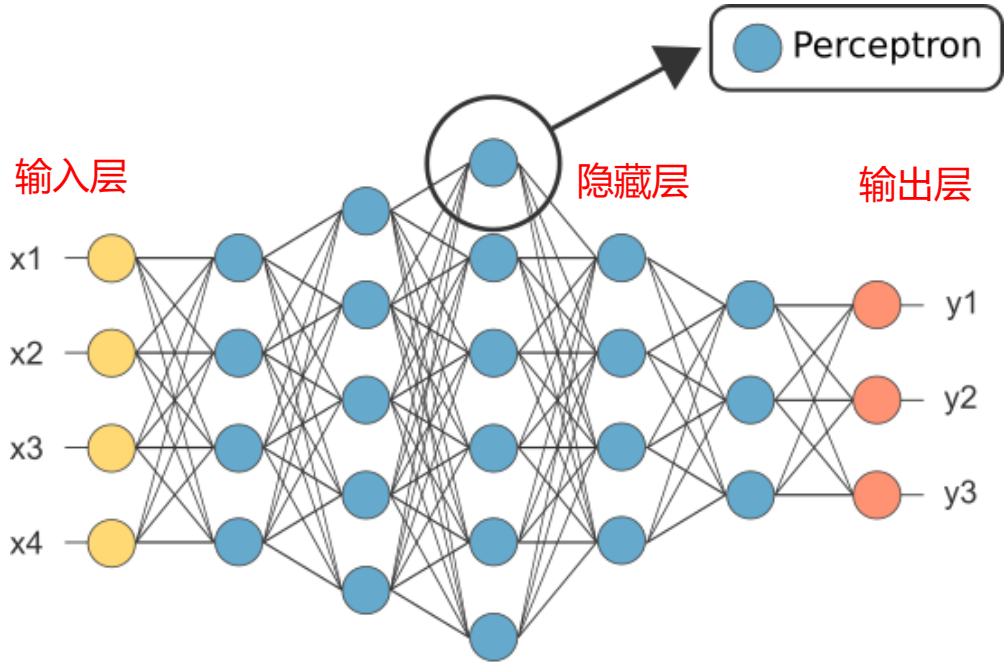
- 机器翻译

$f($ “你好！” $)=$ “Hello!”



视觉皮层的层次结构　　人工神经网络的层次结构　　信号处理的层次结构

全连接神经网络（多层感知机）

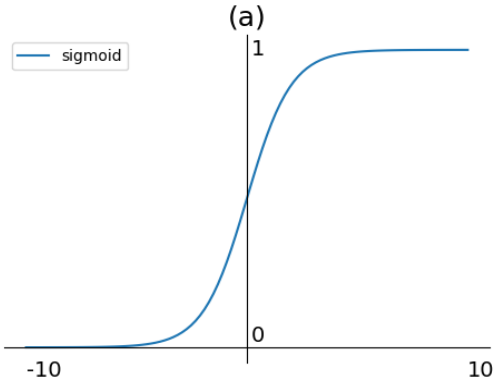


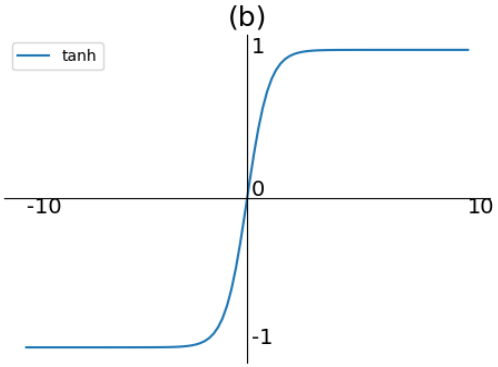
1. 复合函数 $f(\boldsymbol{x}) = f^{(3)}(f^{(2)}(f^{(1)}(\boldsymbol{x})))$
2. 激活函数
3. 梯度下降-反向传播算法

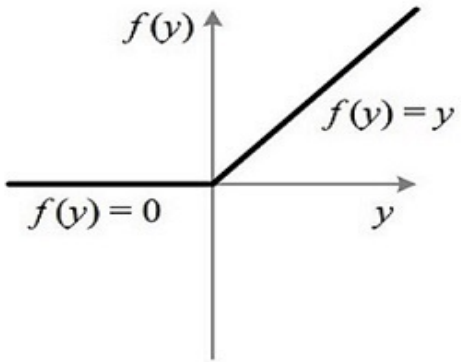
激活函数

激活函数	使用次数	最受欢迎的使用该激活函数的 repo (stars)
ReLU	3,528,912	tensorflow/models (67.5k)
Sigmoid	1,704,283	keras-team/keras (52.1k)
Tanh	876,054	pytorch/examples (46.2k)
Softmax	665,321	tensorflow/models (67.5k)
LeakyReLU	184,298	pytorch/vision (11.2k)

激活函数







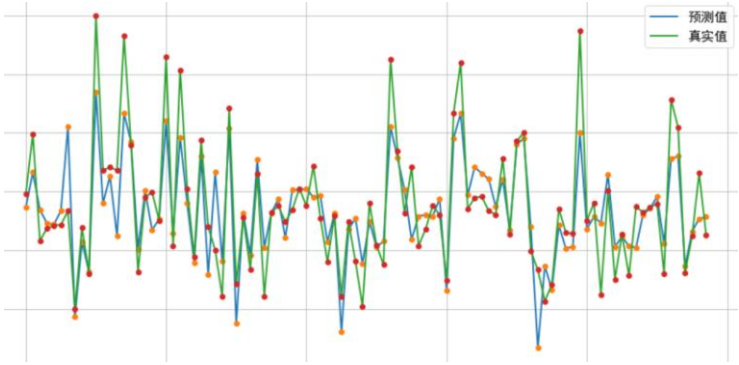
$$sigmoid(x) = \frac{1}{1+e^{-x}} \qquad tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}} = 2sigmoid(2x) - 1 \qquad ReLU(x) = \begin{cases} x & if x > 0 \\ 0 & if x \le 0 \end{cases}$$

$$sigmoid'(x) = sigmoid(x)(1 - sigmoid(x)) \qquad tanh'(x) = 1 - tanh^{2}(x)$$

```
import numpy as np
def sigmoid(x):
    return 1/(1+np.exp(-x))
```

```
import numpy as np
def tanh(x):
    return (np.exp(x)-np.exp(-x))/(np.exp(x)+np.exp(-x))
```

监督学习和损失函数(Loss Function)



回归　　sigmoid　　$$J(\theta_0,\theta_1)=\frac{1}{2m}\sum_{i=1}^{m}(y_i-\hat{y}_i)^2$$

分类　　softmax　　$$H(x,y)=-\sum_{i=i}^{n}x_i\ \ln y_i$$

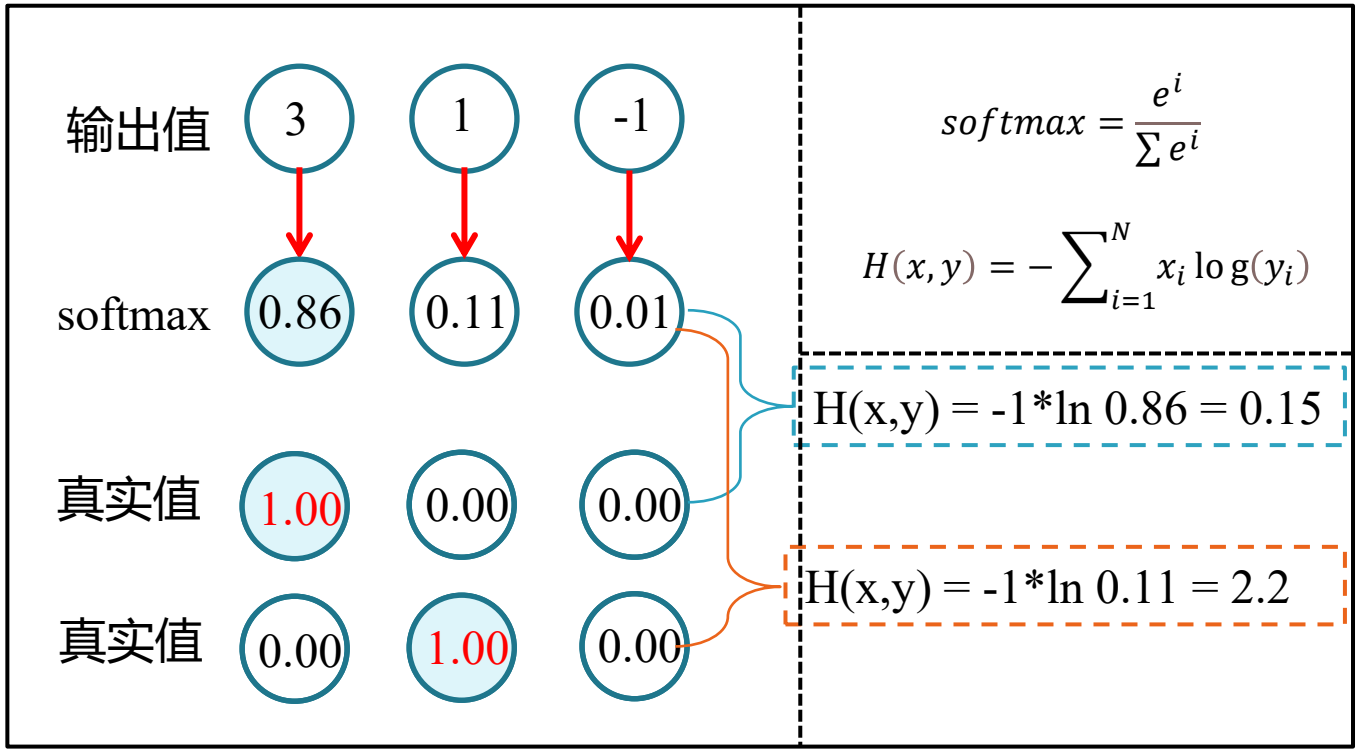
$$S_i=\frac{e^{V_i}}{\sum_j e^{V_j}}$$

One-hot编码

小学 -> [1, 0, 0, 0, 0]
中学 -> [0, 1, 0, 0, 0]
大学 -> [0, 0, 1, 0, 0]
硕士 -> [0, 0, 0, 1, 0]
博士 -> [0, 0, 0, 0, 1]

y_0	0
y_1	1
y_2	0
y_3	0
y_4	0
y_5	0
y_6	0
y_7	0
y_8	0
y_9	0

损失函数(Loss Function) - 交叉熵



Thank you! Questions?